CprE 3810: Computer Organization and Assembly-Level Programming

Project Part 1 Report

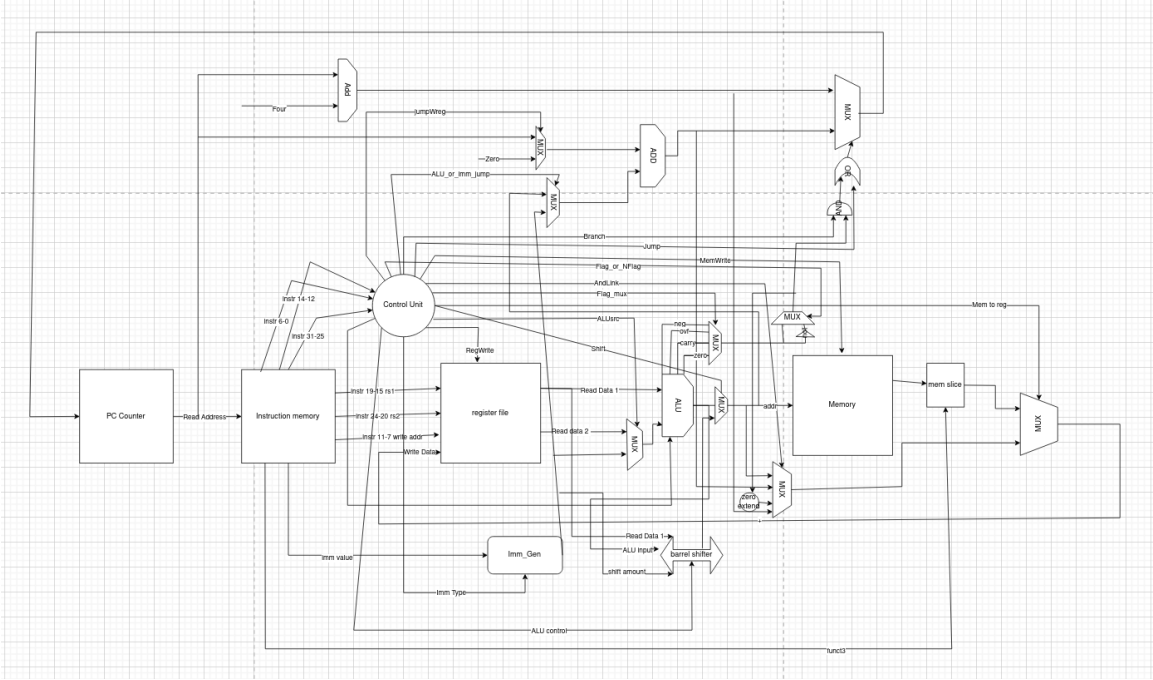
Team Members: Colten Stevens

Graeme Anderson

Project Teams Group #: C_01

Refer to the highlighted language in the project 1 instruction for the context of the following questions.

[Part 2 (d)] Include your final RISC-V processor schematic in your lab report.

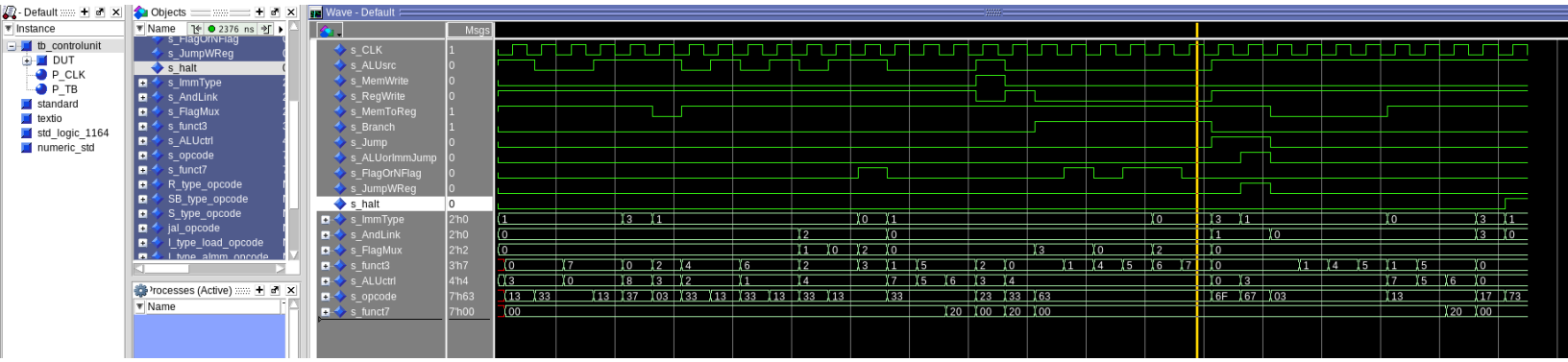


[Part 3.1.a.] Create a spreadsheet detailing the list of *M* instructions to be supported in your project alongside their binary opcodes and funct fields, if applicable. Create a separate column for each binary bit. Inside this spreadsheet, create a new column for the *N* control signals needed by your datapath implementation. The end result should be an *N***M* table where each row corresponds to the output of the control logic module for a given instruction.

	A	B	C	D	E	F	G	H	I	J	K	L	M
1									Control Signals				
2	Instruction	Opcode (Binary)	Funct3	Funct7	ALUSrc (1 bit)	ALUControl (4 bit)	ImmType (2 bit)	AndLink (2 bit)	MemWrite (1 bit)	RegWrite (1 bit)	MemToReg (1 bit)	Branch (1 bit)	Jump (1 bit)
3	addi	0010011	000	XXXXXXX	1	0011	01	00	0	1	1	0	0
4	add	0110011	000	0000000	0	0011	XX	00	0	1	1	0	0
5	and	0110011	111	0000000	0	0000	XX	00	0	1	1	0	0
6	andi	0010011	111	XXXXXXX	1	0000	01	00	0	1	1	0	0
7	lui	0110111	XXX	XXXXXXX	1	1000	11	00	0	1	1	0	0
8	lw	0000011	010	XXXXXXX	1	0011	01	00	0	1	0	0	0
9	xor	0110011	100	0000000	0	0010	XX	00	0	1	1	0	0
10	xori	0010011	100	XXXXXXX	1	0010	01	00	0	1	1	0	0
11	or	0110011	110	0000000	0	0001	XX	00	0	1	1	0	0
12	ori	0010011	110	XXXXXXX	1	0001	01	00	0	1	1	0	0
13	slt	0110011	010	0000000	0	0100	XX	10	0	1	1	0	0
14	slti	0010011	010	XXXXXXX	1	0100	01	10	0	1	1	0	0
15	sltiu	0010011	011	XXXXXXX	1	0100	00	10	0	1	1	0	0
16	sll	0110011	001	0000000	0	0111	XX	00	0	1	1	0	0
17	srl	0110011	101	0000000	0	0101	XX	00	0	1	1	0	0
18	sra	0110011	101	0100000	0	0110	XX	00	0	1	1	0	0
19	sw	0100011	010	XXXXXXX	1	0011	01	XX	1	0	1	0	0
20	sub	0110011	000	0100000	0	0100	XX	00	0	1	1	0	0
21	beq	1100011	000	XXXXXXX	0	0100	01	XX	0	0	1	1	0
22	bne	1100011	001	XXXXXXX	0	0100	01	XX	0	0	1	1	0
23	blt	1100011	100	XXXXXXX	0	0100	01	XX	0	0	1	1	0
24	bge	1100011	101	XXXXXXX	0	0100	01	XX	0	0	1	1	0
25	bltu	1100011	110	XXXXXXX	0	0100	00	XX	0	0	1	1	0
26	bgeu	1100011	111	XXXXXXX	0	0100	00	XX	0	0	1	1	0
27	jal	1101111	XXX	XXXXXXX	X	XXXX	11	01	0	1	1	0	1
28	jalr	1100111	xxx	XXXXXXX	1	0011	01	01	0	1	1	0	1
29	lb	0000011	000	XXXXXXX	1	0011	01	00	0	1	0	0	0
30	lh	0000011	001	XXXXXXX	1	0011	01	00	0	1	0	0	0
31	lbu	0000011	100	XXXXXXX	1	0011	01	00	0	1	0	0	0
32	lhu	0000011	101	XXXXXXX	1	0011	01	00	0	1	0	0	0
33	slli	0010011	001	0000000	1	0111	00	00	0	1	1	0	0
34	srli	0010011	101	0000000	1	0101	00	00	0	1	1	0	0
35	srai	0010011	101	0100000	1	0110	00	00	0	1	1	0	0
36	auipc	0010111	XXX	XXXXXXX	X	XXXX	11	11	0	1	1	0	0
37	wfi	1110011	XXX	XXXXXXX	X	XXXX	XX	XX	X	X	X	X	X

	A	B	C	N	O	P	Q	R
1								
2	Instruction	Opcode (Binary)	Funct3	ALU_Or_Imm_Jump (1 bit)	Flag_Mux (2 bits)	Flag_Or_Nflag	JumpWreg	Halt
3	addi	0010011	000	0	XX	X	0	0
4	add	0110011	000	0	XX	X	0	0
5	and	0110011	111	0	XX	X	0	0
6	andi	0010011	111	0	XX	X	0	0
7	lui	0110111	XXX	0	XX	X	0	0
8	lw	0000011	010	0	XX	X	0	0
9	xor	0110011	100	0	XX	X	0	0
10	xori	0010011	100	0	XX	X	0	0
11	or	0110011	110	0	XX	X	0	0
12	ori	0010011	110	0	XX	X	0	0
13	slt	0110011	010	0	00	0	0	0
14	slti	0010011	010	0	00	0	0	0
15	sltiu	0010011	011	0	00	1	0	0
16	sll	0110011	001	0	XX	X	0	0
17	srl	0110011	101	0	XX	X	0	0
18	sra	0110011	101	0	XX	X	0	0
19	sw	0100011	010	0	XX	X	0	0
20	sub	0110011	000	0	XX	X	0	0
21	beq	1100011	000	1	11	0	0	0
22	bne	1100011	001	1	11	1	0	0
23	blt	1100011	100	1	00	0	0	0
24	bge	1100011	101	1	00	1	0	0
25	bltu	1100011	110	1	00	1	0	0
26	bgeu	1100011	111	1	00	0	0	0
27	jal	1101111	XXX	1	XX	X	0	0
28	jalr	1100111	xxx	0	XX	X	1	0
29	lb	0000011	000	0	XX	X	0	0
30	lh	0000011	001	0	XX	X	0	0
31	lbu	0000011	100	0	XX	X	0	0
32	lhu	0000011	101	0	XX	X	0	0
33	slli	0010011	001	X	XX	X	0	0
34	srli	0010011	101	X	XX	X	0	0
35	srai	0010011	101	X	XX	X	0	0
36	auipc	0010111	XXX	1	XX	X	0	0
37	wfi	1110011	XXX	X	XX	X	X	1

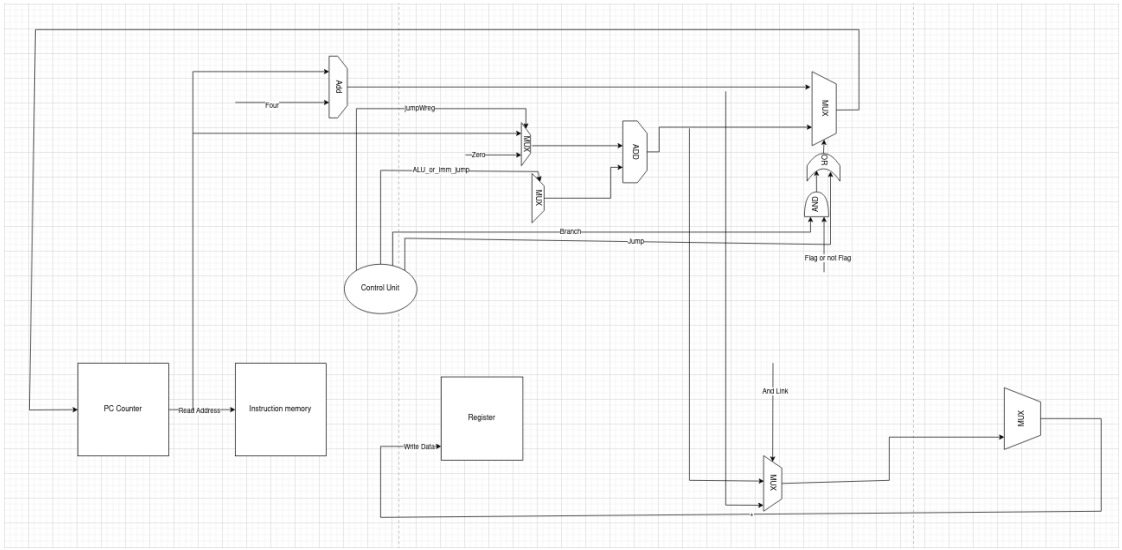
[Part 3.1.(b)] Implement the control logic module using whatever method and coding style you prefer. Create a testbench to test this module individually and show that your output matches the expected control signals from problem 1(a).



[Part 3.2. (a)] What are the control flow possibilities that your instruction fetch logic must support? Describe these possibilities as a function of the different control flow-related instructions you are required to implement.

The control flow logic must be able to preform “JAL” which adds an immediate to the current value of pc and stores the return address of pc+4, “JALR” which takes in a a immediate and a register file adding them together to set that as the pc and stores the return address of pc+4, “AUIPC” sets the pc value to itself plus and immediate and stores that sum to the rd value in registers. Then you have you branching instructions these compute a flag value using two register reads and the ALU sub. Based one the flag value of these computed flags the pc will be updated by being added to by a immediate value determined by the compiler. Lastly in a R-type instruction the PC will just be incremented using an adder by 4.

[Part 3.2. (b)] Draw a schematic for the instruction fetch logic and any other datapath modifications needed for control flow instructions. What additional control signals are needed?



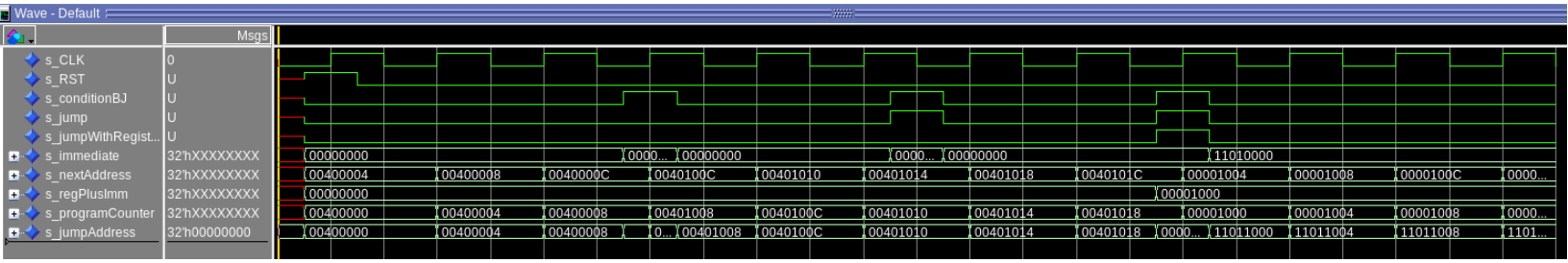
For the fetch implementation we added several control signals as well as logic gates. To start off the control signal ALU or iMM jump determines if you are feeding the immediate or the output of the ALU to be added to the pc.

JumpwReg is a control signal used specifically for “JALR” to calculate the the new pc for a jalr instruction you need to set the value of rs1 + an immediate to the pc. Rs1 + imm is calculated in the ALU fed to the “ALU or imm jump” MUX then it needs to be added with 0 so it is set as just itself to pc and not rs1+imm+pc.

Another thing we did for the fetch logic is increased the size of the and link mux (at the bottom) this allows us to then use the pre existing write back to write either pc+4 or PC +imm back to register rd for auipc and jal instructions.

Lastly for the logic to change pc or to leave it as just pc+4 for the write back of pc. We added a “or” gate as well as a “and” gate to a mux so that all jump instructions update the value of pc but then only branches that fit the specific criteria cause a update to pc.

[Part 3.2.(c)] Implement your new instruction fetch logic using VHDL. Use QuestaSim to test your design thoroughly to make sure it is working as expected. Describe how the execution of the control flow possibilities corresponds to the QuestaSim waveforms in your writeup.



[Part 3.3.1.(a)] Describe the difference between logical (`srl`) and arithmetic (`sra`) shifts. Why does RISC-V not have a `sla` instruction?

A logical shift right (SRL) moves bits to the right and fills the leftmost bits with zeros, which is used for unsigned values. An arithmetic shift right (SRA) also moves bits to the right but fills the leftmost bits with the original sign bit, preserving the sign for signed values.

In contrast, shifting left is the same for both signed and unsigned numbers because bits are simply moved left and zeros are filled on the right. SLA is not implemented because SLL represents multiplying the value by 2 per bit shift. Multiplication by 2 would always result in a zero added on the left, regardless if it is signed or unsigned.

[Part 3.3.1.(b)] In your writeup, briefly describe how your VHDL code implements both the arithmetic and logical shifting operations.

Our barrel shifter implements arithmetic and logical shifting through a few combinations of methods and adjustments. Initially, the barrel shifter sliced the 2 least significant bits of ALUctrl to choose between SLL, SRL, SRA, and pass-through operations.

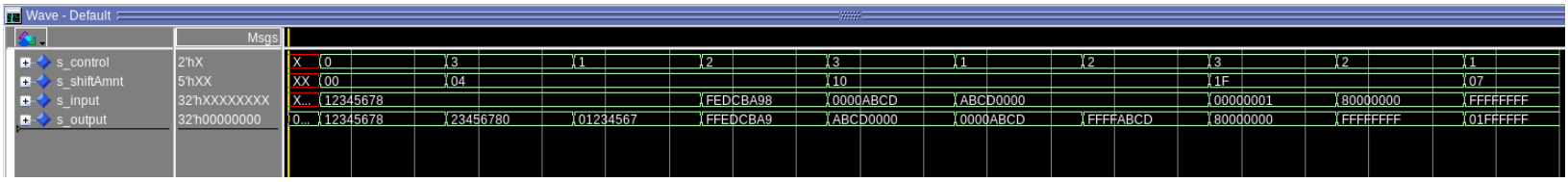
The structure of the barrel shifter is divided into 5 stages of 32 bit and 4x32 bit multiplexers. ‘op’, which represents the slice of ALUctrl is what handles the actual bit adjustments. The 4x32 bit mux will choose between the options of: SRL, SLL, SRA, and pass-through. (11, 10, 01, 00)

Stage 0 will shift by 1 bit, Stage 1 by 2, and so on up to a Stage 4 which shifts by 16. This allows for total coverage of the 32 bit shifts (0 to 31). Each stage is selected by ‘shamt’ (Shift Amount) which is a 5 bit vector that selects the 32 bit mux to either use the shifted value or pass-through the previous stage, indicating no shift at that stage.

[Part 3.3.1.(c)] In your writeup, explain how the right barrel shifter above can be enhanced to also support left shifting operations.

The Right barrel shifter, if given options for SRL, SRA, and pass-through, requires 4 input multiplexers with 1 empty input. This empty input can be used for SLL instructions, with the appropriate slices and ‘0’ bit concatenation added on. It takes advantage of the empty input to maximize hardware usage in implementation.

[Part 3.3.1.(d)] Describe how the execution of the different shifting operations corresponds to the QuestaSim waveforms in your writeup.



Shifts in Order:

1. No Shift — Input = Output
2. SLL by 4 — Input loses highest hex value, adds a 0 to the least hex value
3. SRL by 4 — Input loses lowest hex value, adds a 0 to the highest hex value
4. SRA by 4 — Input loses lowest hex value, adds an F to the highest hex value
5. SLL by 16 — Input moves bottom half to top half, replaces bottom with 0s
6. SRL by 16 — Input moves top half to bottom half, replaces top with all 0s.
7. SRA by 16 — Input moves top half to bottom half, replaces top with all 1s (Fs).
8. SLL by 31 — Maximum shift
9. SRA by 31 — Maximum Shift
10. SLL by 7 — Shift left by 7 (arbitrary amount)

[Part 3.3.2.(a)] In your writeup, briefly describe your design approach, including any resources you used to choose or implement the design. Include at least one design decision you had to make.

For our design approach of the ALU we wanted to basically run every sign type of ALU instruction at 1 time and then just mux the outputs of these different components to get our desired result. Along with this, for every single instruction we calculate 5 different flag values (ovf, neg, carry, slt_flag (ovf and neg flag xored), and zero). These flags can also be muxed through inside ALU depending on the ALU control signals given to the ALU.

Flag_mux table:

00= neg flag

01= SLT flag

10= carry flag

11 = zero flag

ALU control table=

and= 0000

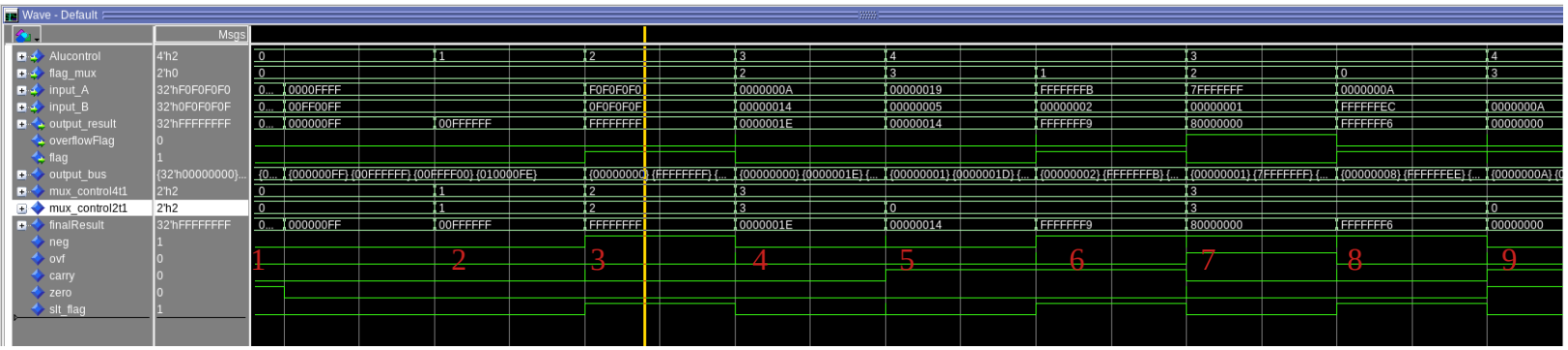
or= 0001

xor=0010

add=0011

sub=0100

[Part 3.3.2.(b)] Describe how the execution of the different operations corresponds to the QuestaSim waveforms in your writeup.

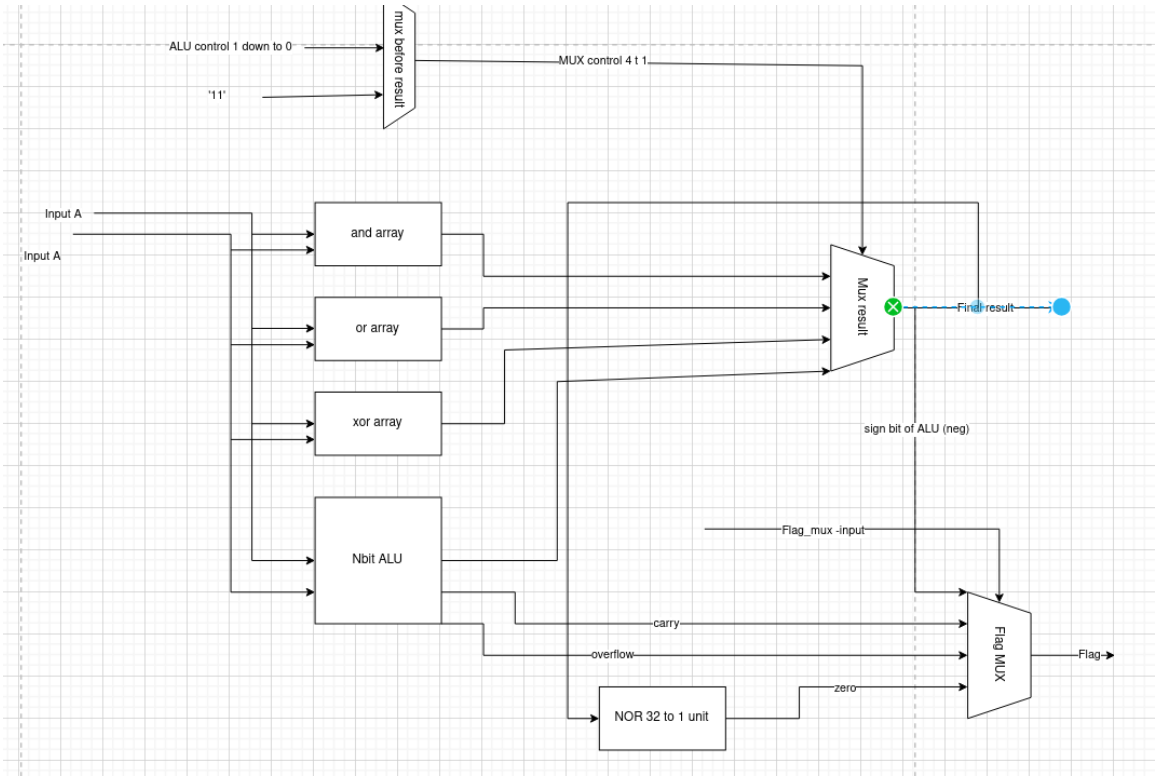


This is our test bench for our ALU

1. and's the two inputs together 0000FFFF And 00FF00FF to get 000000FF
2. ors the two inputs together 0000FFFF or 00FF00FF to get 00FFFFFF
3. xor's the two inputs together F0F0F0F0 xor 0F0F0F0 to get FFFFFFFF
4. adds the two inputs together 0000000A add 00000014 to get 0000001E
5. subs the two inputs together 00000019 sub 00000005 to get 00000014
6. looking for SLT flag -5= rs1 and 2= rs2 because rs1<rs2 slt is true and flag is 1.
7. overflow and carry adds rs1= 7FFFFFFF add rs2= 00000001 overflawflag set to 1 and flag looking for carry also 1. (OVERFLOW NOT IN RARS BUT CONCEPT CHECK)

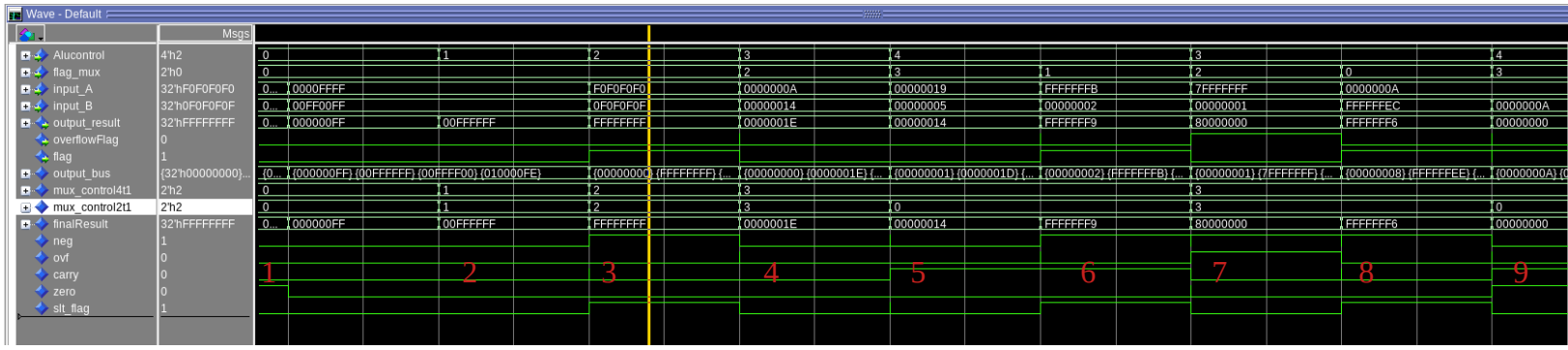
8. negative flag test 10 add with -20. rs1 = 0000000A add rs2 = FFFFFFEC output = FFFFFFF6 (negative 10) so flag is set to 1.
9. sub rs1= 0000000A and rs2 =0000000A output 00000000 flag set to 1 looking for neg flag

[Part 3.3.3] Draw a simplified, high-level schematic for the 32-bit ALU. Consider the following questions: How is Zero calculated? How is `slt` implemented?



The zero flag is calculated by a nor tree unit, this unit takes the bit slices of the add/sub unit input ors them all together and then at the very end after oring down to 1 bit nots that last bit. This means that if the final value held any value that zero flag would be false.

[Part 3.3.5] Describe how the execution of the different operations corresponds to the QuestaSim waveforms in your writeup.



This is our test bench for our ALU
1. and's the two inputs together 0000FFFF And 00FF00FF to get 000000FF
2. ors the two inputs together 0000FFFF or 00FF00FF to get 00FFFFFF
3. xor's the two inputs together F0F0F0F0 xor 0F0F0F0 to get FFFFFFFF
4. adds the two inputs together 0000000A add 00000014 to get 0000001E
5. subs the two inputs together 00000019 sub 00000005 to get 00000014
6. looking for SLT flag -5= rs1 and 2= rs2 because rs1<rs2 slt is true and flag is 1.
7. overflow and carry adds rs1= 7FFFFFFF add rs2= 00000001 overflawflag set to 1 and flag looking for carry also 1. (OVERFLOW NOT IN RARS BUT CONCEPT CHECK)
8. negative flag test 10 add with -20. rs1 = 0000000A add rs2 = FFFFFFEC output = FFFFFFF6 (negative 10) so flag is set to 1.
9. sub rs1= 0000000A and rs2 =0000000A output 00000000 flag set to 1 looking for neg flag

[Part 3.3.8] justify why your test plan is comprehensive. Include waveforms that demonstrate your test programs functioning.
This test plan is comprehensive because it tests outputs for every function of the ALU (and,or,xor,add,sub, and all flags that are used for branch instructions)

[Part 4] In your writeup, show the QuestaSim output for each of the following tests, and provide a discussion of result correctness. It may be helpful to also annotate the waveforms directly.

Shown in past 2 questions

[Part 4.a] Create a test application that makes use of every required arithmetic/logical instruction at least once. The application need not perform any particularly useful task, but it should demonstrate the full functionality of the processor (e.g., sequences of many instructions executed sequentially, 1 per cycle while data written into registers can be effectively retrieved and used by later instructions). Name this file Proj1_base_test.s.
[Part 4.b] Create and test an application which uses each of the required control-flow instructions and has a call depth of at least 5 (i.e., the number of activation records on the stack is at least 4). Name this file Proj1_cf_test.s.

[Part 4.c] Create and test an application that sorts an array with *N* elements using the MergeSort algorithm (link). Name this file Proj1_mergesort.s.

All 3 applications are in the submission.

[Part 5] Report the maximum frequency your processor can run at and determine what your critical path is. Draw this critical path on top of your top-level schematic from part 1. What components would you focus on to improve the frequency?

Frequency Maximum as reported in synthesis is 25.43 MHz.

Our Critical Path is as such:

Fetch Unit → Instruction Memory → Register File → ALU → Fetch Unit → Register File.

This is indicative of a JALR instruction, where we utilize 2 separate ripple-carry adders in the ALU to determine the Register + Immediate value, and then inside of the fetch unit we use another to determine (Reg+Imm) + PC.

To reduce bottlenecks and improve the frequency, we should change the ALU adder and the Fetch Unit adders to be faster and more advanced designs such as a carry-lookahead adder.

